

Assignment 3

MA546: Introduction to Quantitative Finance

Q1. Binary Encodings

What is binary encoding? Explain how portfolio weights or investment decisions can be represented using binary strings.

Binary encoding means presenting decisions using binary symbols aka. 0s and 1s. In portfolio optimization (using Genetic Algorithms), each bit can represent:

- Whether to invest or not
- The level of allocation
- A discretized value of weight

Example: For a portfolio with 6 assets, the binary string 101010 can be interpreted as:

- **1:** Invest in asset
- **0:** Do not invest

So, 101010 = Invest in assets 1, 3, 5, and do not invest in assets 2, 4, 6.

We can also encode percent allocation. For example, the binary string 011001 equals 25 in decimal, which could represent a 25% portfolio allocation or means we have discrete allocation levels (e.g., 0%, 10%, 20%, ..., 100%), each level can be mapped to a binary string. For example, with 4 bits $\rightarrow$ 16 levels and divide our asset to have allocation in one of this level only.

A chromosome can be a concatenation of binary strings for each asset weight. Investment decisions (e.g., buy/sell/hold) can also be encoded as binary flags.

Q2. Gray Codes

What is Gray code? Explain how it is constructed from binary representation and why it might be used in optimization of portfolios.

Gray code is a binary numbering system where **only one bit changes between consecutive numbers**. This reduces sudden jumps in optimization algorithms.

How Gray Code is Constructed:

- The first bit remains the same.
- For the next bits: $Gray_bit = Previous_Binary_bit \oplus Current_Binary_bit$ (XOR).

Why it is used in Portfolio Optimization: It prevents large jumps in values, makes the search landscape smoother, and improves convergence stability. It is especially useful when portfolio weights are continuous and the optimization requires gradual changes.

In Current binary encoding, a small change in integer like 3 to 4 will change all the bits showing very large change in phenotype. Gray code ensures small genotypic changes correspond to small phenotypic changes, making local search smoother.

Q3. Difference Between Binary and Gray Encoding

Discuss in terms of (a) bit changes between consecutive numbers, and (b) Hamming distance.

(a) Bit Changes Between Consecutive Numbers

- **Binary:** Going from 3 (011) to 4 (100) requires changing **all 3 bits**.
- **Gray:** Going from 3 (010) to 4 (110) requires changing **only 1 bit**.

(b) Hamming Distance The Hamming distance represents the number of bits that are different between two strings.

System	Typical Distance (Consecutive Integers)
Binary	Large / Variable (can be up to all bits)
Gray	Always 1

Conclusion: Gray code provides a smoother search, whereas Binary is faster to decode but highly jumpy.

Q4. Convert Binary $\rightarrow$ Gray and Decimal

Convert the following binary strings to Gray code and compute their integer values, representing portfolio allocation levels.

Solution	Binary	Gray	Integer (Base 10)
S_1	000101	000111	5
S_2	001111	001000	15
S_3	010010	011011	18

Step-by-step example for S_1 (000101):

$g_1 = 0$, $g_2 = 0 \oplus 0 = 0$, $g_3 = 0 \oplus 0 = 0$, $g_4 = 0 \oplus 1 = 1$, $g_5 = 1 \oplus 0 = 1$, $g_6 = 0 \oplus 1 = 1$.
Result: 000111.

Q5. Convert Gray $\rightarrow$ Binary and Decimal

Convert the following Gray code to binary and compute their integer values.

Rule: $Binary_bit = Previous_Binary_bit \oplus Gray_bit$

Solution	Gray	Binary	Integer (Base 10)
S_4	111010	101100	44
S_5	101101	110110	54
S_6	010111	011010	26

Q6. Portfolio Allocation, Fitness, and Roulette Selection

Compute fitness, probabilities, and execute selections.

(a, b, c) Fitness and Probabilities

Using the fitness function $f(x) = x^2$. Total Fitness = $100+625+900+2025+2500+3600 = 9750$.

Sol.	Alloc (%)	Fitness $f(x)$	Probability	Cumulative Prob.
S_1	10	100	0.0103	0.0103
S_2	25	625	0.0641	0.0744
S_3	30	900	0.0923	0.1667
S_4	45	2025	0.2077	0.3744
S_5	50	2500	0.2564	0.6308
S_6	60	3600	0.3692	1.0000

(d) Roulette-Wheel Selection

Random r	0.12	0.35	0.58	0.77	0.89	0.95
Selected	S_3	S_4	S_5	S_6	S_6	S_6

(e) Tournament Selection (size=2)

Winner is the solution with the higher fitness.

Tournament	Participants	Winner
T_1	(S_1, S_4)	S_4
T_2	(S_2, S_5)	S_5
T_3	(S_3, S_6)	S_6
T_4	(S_1, S_3)	S_3
T_5	(S_2, S_4)	S_4
T_6	(S_5, S_6)	S_6

Q7. Crossover and Mutation for Binary GA

Perform various crossover operations.

(a) One-point crossover at position 3:

$P_1 = 101|101, P_2 = 011|010 \Rightarrow O_1 = 101010, O_2 = 011101$

(b) Two-point crossover (positions 2 to 5):

$P_1 = 1|1001|1, P_2 = 0|0110|1 \Rightarrow O_1 = 101101, O_2 = 010011$

(c) Uniform crossover with mask $M = 110010$:

$P_1 = 101010, P_2 = 010101 \Rightarrow O = 100111$

(d) Identify parents given offspring (1-point crossover at pos 3):

$O_1 = 101010, O_2 = 011101 \Rightarrow P_1 = 101101, P_2 = 011010$

(e) Perform crossover and mutation:

$P_1 = 111000, P_2 = 000111$. Crossover at pos 3 $\Rightarrow O_1 = 111111, O_2 = 000000$.

Mutate bit 2 of O_1 ($1 \rightarrow 0$) $\Rightarrow$ **Final:** 101111.

Q8. Real-Coded GA and Inverse CDF Sampling

Explain inverse transform method and generate offspring.

(a) Inverse Transform Method:

Generates a uniform random number $u \sim U(0,1)$ and applies the inverse CDF: $x = F^{-1}(u)$. This converts uniform random numbers into the desired distribution, generating realistic offspring while preserving diversity.

(b) Generate offspring using Inverse CDF:

Given $F(x) = x^2$, the inverse is $x = \sqrt{u}$.

Uniform Random u	Offspring $x = \sqrt{u}$
0.1	0.316
0.25	0.500
0.5	0.707
0.75	0.866
0.9	0.949

(c) Polynomial Mutation:

Given: $x = 0.4, x_{min} = 0, x_{max} = 1, n_m = 2$.

Formula: $\delta = (2u)^{1/3} - 1$, and mutated offspring $x' = x + \delta$.

Random u	δ	Mutated Offspring x'
0.2	-0.263	0.137
0.5	0.000	0.400
0.8	0.170	0.570